RNGD_SSH_uv_init_

RNGD NPU 서버: 2인 1조 독립 실습환경 구축

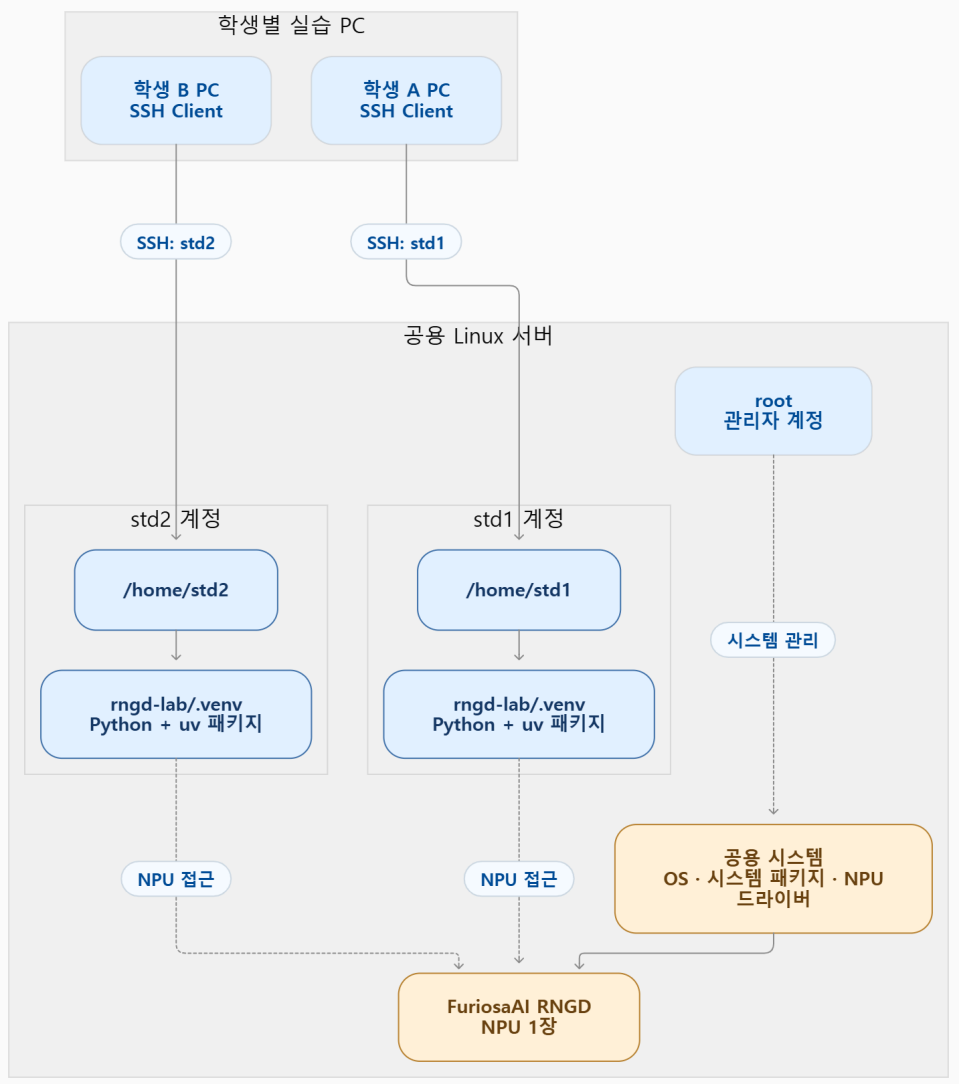
> **실습 주제:** Linux 사용자 계정 분리 → SSH 접속 → sudo 권한 확인 → `uv init` 프로젝트 구성 → Python 패키지 관리 → RNGD 장치 확인
>
> **대상:** RNGD NPU 1장이 할당된 2인 1조 교육생
>
> **환경 가정:** Ubuntu/Debian 계열 Linux, SSH 접속 가능, RNGD 드라이버 및 SDK는 관리자 사전 설치
>
> **계정:** 관리자 `root`, 교육생 `std1` 및 `std2`
>
> **중요:** 서버 운영체제와 RNGD 관련 도구의 실제 설치 상태는 사전에 확인한다.

1. 학습 목표

실습 완료 후 교육생은 다음 작업을 수행할 수 있다.

1. 하나의 Linux 서버에서 사용자별 SSH 계정과 홈 디렉터리를 구분한다.
2. `whoami`, `id`, `groups`, `sudo whoami` 명령으로 계정 및 권한을 확인한다.
3. **계정별로 uv를 설치하고 `uv init`으로 독립 Python 프로젝트를 생성한다.**
4. `uv add`, `uv sync`, `uv run`으로 의존성과 실행환경을 관리한다.
5. 계정별 Python 패키지 환경과 공용 RNGD 장치의 차이를 설명한다.

2. 실습환경 아키텍처

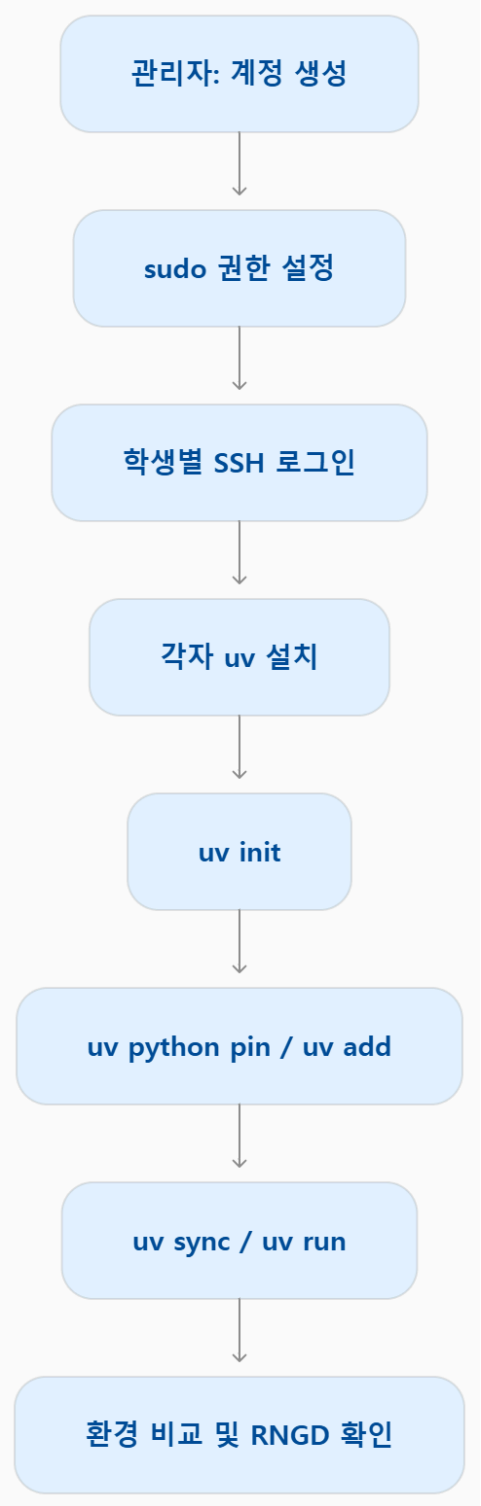


2.1 무엇이 분리되고 무엇이 공유되는가?

항목	분리 여부	비고
SSH 로그인 세션 / 사용자 UID	분리	`std1`, `std2` 서로 다른 계정
홈 디렉터리	분리	`/home/std1`, `/home/std2`
uv 설치	분리 가능	각각 사용자 홈에 설치
uv 프로젝트 / `.venv` / Python 패키지	분리	프로젝트별 관리
시스템 Python / `apt` 패키지	공유	서버 전체에 적용
CPU / 시스템 RAM / 파일 시스템 장치	공유	완전한 컨테이너 격리는 아님
RNGD 드라이버 / RNGD 물리 카드	공유	동시 실행 시 자원 경합 가능
sudo 관리자 권한	부여 가능	**전체 sudo 권한은 보안 격리가 아님**

3. 실습 흐름

▼ 실습 실행 흐름



---

4. 실습 1 — 관리자: 계정 생성 및 SSH 접속 준비

> **실행 주체: 서버 관리자(root)**

> 이미 `std1`, `std2`가 생성되어 있다면 계정 생성 명령은 생략한다.

▼ Step 1. 운영체제 확인

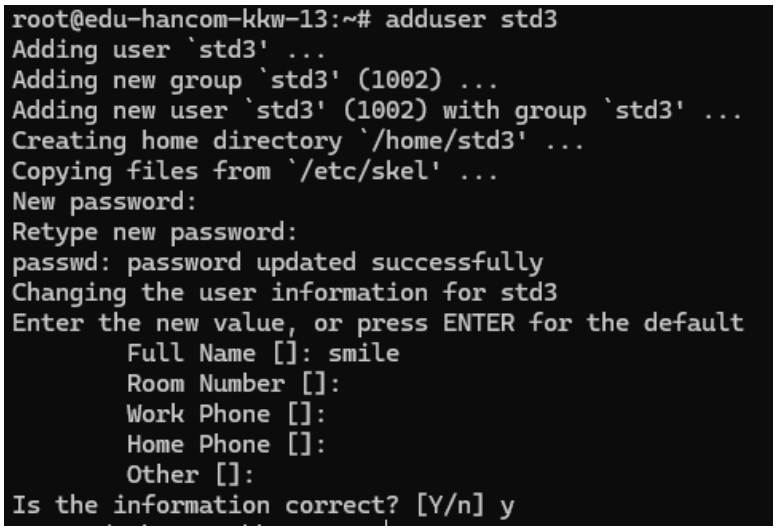
```
cat /etc/os-release
```

다음 `apt`, `adduser`, `usermod`, `sudo` 명령은 Ubuntu/Debian 계열을 기준으로 한다.

▼ Step 2 사용자 생성

```
adduser std1
adduser std2
```

`adduser` 실행 중 각 사용자 비밀번호를 설정한다. 이미 존재하는 계정은 다시 생성하지 않는다.



▼ Step 3 사용자 및 홈 디렉터리 확인

```
id std1
id std2
ls -ld /home/std1 /home/std2
```

서로 다른 UID와 홈 디렉터리가 나타나야 한다.

```
root@edu-hancom-kkw-13:~# id std3
uid=1002(std3) gid=1002(std3) groups=1002(std3)
root@edu-hancom-kkw-13:~# ls -ld /home/std3
drwxr-x--- 2 std3 std3 4096 Oct 17 22:51 /home/std3
root@edu-hancom-kkw-13:~# id std1
uid=1000(std1) gid=1000(std1) groups=1000(std1),27(sudo)
root@edu-hancom-kkw-13:~# ls -ld /home/std1 /home/std2
drwxr-x--- 6 std1 std1 4096 Oct 17 06:20 /home/std1
drwxr-x--- 6 std2 std2 4096 Oct 17 06:20 /home/std2
root@edu-hancom-kkw-13:~#
```

▼ Step 4. sudo 설치 및 권한 부여

```
command -v sudo
```

설치 되어 있지않다면,

```
root@edu-hancom-kkw-12:~# command -v sudo
root@edu-hancom-kkw-12:~#
```

설치 되어 있다면,

```
root@edu-hancom-kkw-13:~# command -v sudo
/usr/bin/sudo
root@edu-hancom-kkw-13:~#
```

sudo가 없는 Ubuntu/Debian 서버에서만, root로 실행한다.

```
apt update
apt install -y sudo
```

실습 정책상 전체 관리자 권한을 허용하는 경우에만 아래 명령을 실행한다.

```
# sudo 권한 부여
usermod -aG sudo std1
usermod -aG sudo std2

# sudo 권한 부여 확인
groups std1
groups std2
```

```
root@edu-hancom-kkw-13:~# usermod -aG sudo std3
root@edu-hancom-kkw-13:~# groups std3
std3 : std3 sudo
root@edu-hancom-kkw-13:~#
```

두 계정에 `sudo` 그룹이 표시되는지 확인한다. 그룹 변경 후에는 학생이 **SSH를 다시 로그인**해야 한다.

▼ 보안주의

> **보안 주의:** 실습용 공유 서버에서는 학생에게 전체 sudo를 주지 않고, 관리자는 OS·드라이버를 관리하며 학생은 자신의 홈 디렉터리에서 개발하게 하는 방식이 더 안전하다. 전체 sudo를 부여하면 다른 사용자의 파일과 시스템 설정에 접근할 수 있다.

> `현재 sudo 권한 제거 방법`

```
gpasswd -d std1 sudo
gpasswd -d std2 sudo
```

▼ Step 5. 학생별 SSH 키 관리

가능하면 `root`, `std1`, `std2`에 **각기 다른 SSH 키 쌍**을 사용한다. 동일한 개인키를 모든 계정에 재사용하면 그 키를 가진 사용자가 접근 가능한 계정까지 함께 접속할 수 있다.

- 공개키: 서버의 각 사용자 `~/.ssh/authorized_keys`에 등록
- 개인키: 해당 사용자 PC에 안전하게 보관

---

5. 실습 2 — 교육생: SSH 로그인 및 권한 확인

> **실행 주체: std1, std2 각각 본인 PC에서 수행**

▼ Step 1. SSH 접속

▼ 접속 방법1 CLI 명령으로 접속하기

root :

```
ssh -i ~/.ssh/id_rsa_<번호> -p <SSH포트> root@eval.nxt.
furiosa.ai
ssh -i id_rsa_13 -p 36140 root@eval.nxt.furiosa.ai
```

학생 A:

```
ssh -i id_rsa_13 -p 36140 std1@eval.nxt.furiosa.ai
```

학생 B:

```
ssh -i id_rsa_13 -p 36140 std2@eval.nxt.furiosa.ai
```

▼ 접속 방법2 config 파일에 설정하여 연결하기

~/.ssh/config 파일 작성

```
# hancom-kkw-13
Host hancom-kkw-13
  HostName eval.nxt.furiosa.ai
  User root
  Port 36140
  IdentityFile C:\Users\magpi\.ssh\id_rsa_13
  IdentitiesOnly yes

# hancom-kkw-13 std3
Host hancom-kkw-13_std3
  HostName eval.nxt.furiosa.ai
  User std3
  Port 36140
  IdentityFile C:\Users\magpi\.ssh\id_rsa_13
  IdentitiesOnly yes
```

서버에 접속하기

```
cd ~/.ssh

# root로 접속
```

```
ssh hancom-kkw-13

# std3
ssh hancom-kkw-13_std3
```

▼ Step 2. 현재 계정 및 디렉터리 확인

```
whoami
pwd
id
groups
```

`std3`로 접속한 경우 기대 결과의 핵심 부분:

```
std3
/home/std3
...
std3 sudo
```

`id`에 출력되는 UID/GID 숫자는 서버마다 다르다.

▼ Step 3. 관리자 권한 확인 (허용된 환경에 한함)

sudo 명령 사용하기

```
sudo whoami
```

출력이 `root`이면 해당 명령을 관리자 권한으로 실행할 수 있다는 뜻이다.

올바른 구문:

```
sudo apt update  # 시스템 패키지 목록 갱신
```

잘못된 예:

```
sudo update  # 잘못된 명령: update라는 실행 파일이 없음
```

> **주의:** `sudo apt update`, `sudo apt upgrade`, `sudo apt install`은 사용자 전용 가상환경 관리 명령이 아니다. 특히 RNGD 드라이버를 사용하는 교육 서버에서는 `apt upgrade`

> 를 학생이 임의로 실행하지 않는다.

---

6. 실습 3 — 교육생(사용자 계정): uv를 사용자 계정별 설치

> **실행 주체: std1과 std2가 각자의 SSH 세션에서 실행**
>
> 아래 명령에는 `sudo`를 붙이지 않는다.

▼ Step 1. uv 설치

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

`curl`이 없거나 외부 인터넷 접근이 제한되어 있으면 강사에게 설치 환경을 확인한다.

▼ Step 2. PATH 설정 적용

```
source ~/.local/bin/env
```

설치 스크립트가 출력한 안내를 우선 적용한다. 재로그인이 필요한 환경도 있다.

▼ Step 3. uv 버전 및 설치 경로 확인

```
uv --version
command -v uv
```

사용자별 설치 경로 예:

```
/home/std1/.local/bin/uv
/home/std2/.local/bin/uv
```

---

7. 실습 4 — `uv init`으로 프로젝트 생성

> **핵심:** 본 교재는 `uv venv` 중심이 아니라 **`uv init`으로 프로젝트를 시작하는 방식**을 사용한다.
>
> `uv init`은 프로젝트 구조와 설정 파일을 생성한다. 프로젝트의 실제 가상환경 `.venv`는 이후 `uv sync`, `uv add`, `uv run` 등의 명령으로 자동 생성·관리한다.

▼ Step 1. 홈 디렉터리에서 프로젝트 생성

```
cd ~
uv init --bare --python 3.12 --pin-python rngd-lab
cd ~/rngd-lab
uv sync
```

이미 `~/rngd-lab`가 존재하고 그 안에 파일이 있다면 덮어쓰지 않는다.

`uv`가 Python 3.12 실행 환경을 찾지 못하면 관리형 Python을 내려받을 수 있다. 실제 다운로드 가능 여부는 서버의 네트워크 정책에 따른다.

▼ Step 2. 프로젝트 생성 결과 확인

```
pwd
ls -la
cat pyproject.toml
ll
```

대표적인 프로젝트 파일:

```
/home/std1/rngd-lab/
├── .python-version
├── .venv/
├── pyproject.toml
└── uv.lock
```

uv 버전과 템플릿에 따라 생성 파일 목록은 달라질 수 있다.

▼ Step 3. 프로젝트 의존성 추가

```
uv add numpy pandas requests
```

프로젝트 의존성이 `pyproject.toml`에 기록되고 `uv.lock` 및 `.venv`가 생성·갱신된다.

▼ Step 4. 의존성 동기화

```
uv sync
```

`uv sync`는 프로젝트 설정과 잠금 파일에 맞춰 가상환경 패키지를 동기화한다.

▼ Step 5. Python 실행

```
uv run python --version
uv run python -c "import sys; print(sys.executable)"
uv run python -c "import numpy, pandas, requests; print
('packages OK')"
```

`uv run`은 해당 프로젝트 가상환경에서 명령을 실행한다. **매번** `source .venv/bin/activate`**를 수행할 필요가 없다.**

▼ Step 6. 가상환경 활성화 (선택)

```
source .venv/bin/activate
which python
python --version
# 종료하려면:
deactivate
```

가상환경 활성화는 선택 사항이다. 수업에서는 `uv run` 방식을 기본으로 사용한다.

---

8. 실습 5 — 계정별 패키지 분리 확인

▼ Step 1. std1에서만 추가 패키지 설치

`std1`에서:

```
cd ~/rngd-lab
uv add rich
uv run python -c "import rich; print('std1: rich import
OK')"
```

▼ Step 2. std2에서는 다른 패키지 설치

`std2`에서:

```
cd ~/rngd-lab
uv add httpx
```

```
uv run python -c "import httpx; print('std2: httpx impor
t OK')"
```

▼ Step 3. 의존성 비교

두 학생 모두:

```
cat pyproject.toml
uv tree
```

`std1`에는 직접 추가한 `rich`, `std2`에는 직접 추가한 `httpx`가 각각 표시된다. 단, 간접 의존성으로 동일한 패키지가 설치될 수 있으므로 **직접 의존성 목록**을 기준으로 비교한다.

▼ Step 4. uv 프로젝트 관리 명령 정리

명령	기능	시점
`uv init rngd-lab --python 3.12`	새 프로젝트 초기화	프로젝트 시작
`uv python pin 3.12`	Python 버전 지정	최초 구성
`uv add pandas`	패키지 의존성 추가	개발 중
`uv remove pandas`	의존성 제거	변경 시
`uv sync`	잠금 파일 기준 동기화	환경 재구성
`uv run python main.py`	프로젝트 환경에서 실행	실습 실행
`uv tree`	의존성 트리 조회	검증
`uv lock`	의존성 잠금 파일 갱신	잠금 관리

---

9. 실습 6 — Python 환경 검증 코드 실행

`std1`, `std2`는 각각 자신의 프로젝트에서 아래 코드를 작성한다.

▼ Step 1. `check_env.py` 작성

```
cd ~/rngd-lab
cat > check_env.py <<'PY'
import getpass
import sys
from pathlib import Path
```

```
print("=== RNGD 실습환경 확인 ===")
print(f"사용자: {getpass.getuser()}")
print(f"홈 디렉터리: {Path.home()}")
print(f"Python 버전: {sys.version.split()[0]}")
print(f"Python 실행 경로: {sys.executable}")
print(f"프로젝트 위치: {Path.cwd()}")
PY
```

▼ Step 2. `uv run` 으로 실행

```
uv run python check_env.py
```

학생 A의 예상 출력:

```
=== RNGD 실습환경 확인 ===
사용자: std1
홈 디렉터리: /home/std1
Python 버전: 3.12.x
Python 실행 경로: /home/std1/rngd-lab/.venv/bin/python
프로젝트 위치: /home/std1/rngd-lab
```

학생 B의 예상 출력:

```
=== RNGD 실습환경 확인 ===
사용자: std2
홈 디렉터리: /home/std2
Python 버전: 3.12.x
Python 실행 경로: /home/std2/rngd-lab/.venv/bin/python
프로젝트 위치: /home/std2/rngd-lab
```

`3.12.x` 는 예시 표기이며 실제 출력에서는 설치된 패치 버전이 표시된다.

▼ Step 3. 조별 결과 비교

- 사용자 이름이 서로 다른가?
- 홈 디렉터리가 서로 다른가?
- Python 실행 경로가 서로 다른가?
- `pyproject.toml` 의 직접 의존성이 서로 다른가?

10. 실습 7 — RNGD NPU 장치 확인

> RNGD 관리 도구가 서버에 사전 설치되어 있어야 한다. 계정별 uv 설치만으로 RNGD 드라이버나 SDK가 자동 설치되지는 않는다.

▼ Step 1. 관리 도구 존재 확인

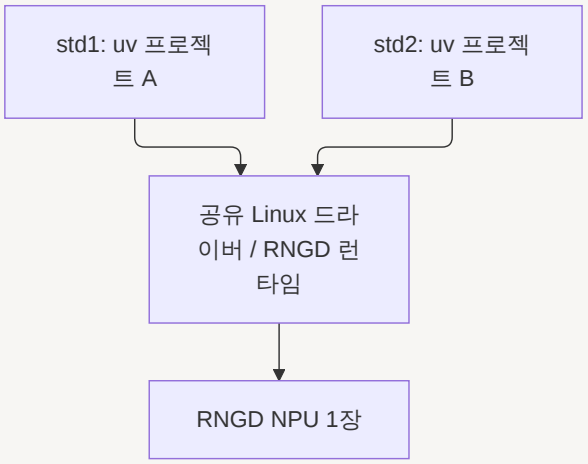
```
command -v furiosa-smi
```

▼ Step 2. 장치 정보 확인

```
furiosa-smi info
```

두 계정에서 정상 조회되는지 확인한다. 장치 조회 성공은 **NPU 접근을 확인하는 첫 단계**이며, 두 사람이 모델을 동시에 독립 로딩할 수 있다는 보장은 아니다.

▼ Step 3. 공용 자원과 독립 자원 구분



- 학생별 독립 수행: 코드 작성, 가상환경, Python 라이브러리 설치, API 클라이언트 개발
- 조별 협의 필요: 모델 로딩, NPU 추론 서버 기동, 포트 사용, 메모리 사용량
- 관리자 담당 권장: OS 및 드라이버 업데이트, NPU 장치 설정

---

11. 주요 오류 및 해결 방법

증상	원인 또는 점검 지점	해결 방법
`sudo: update: command not found`	`sudo update`는 잘못된 명령	`sudo apt update`가 올바른 구문
`sudo: command not found`	sudo 미설치 또는 경로 문제	관리자에게 설치 확인 요청
`is not in the sudoers file`	sudo 권한 미부여	강사에게 권한 확인 요청
`uv: command not found`	uv 경로가 셸에 미적용	`source ~/.local/bin/env`, 재접속
`No such file or directory: .venv`	`uv init` 후 아직 환경 미생성	`uv sync` 실행
Python 3.12 미발견	Python 미설치 / 다운로드 제한	`uv python install 3.12` 또는 관리자 확인
`ModuleNotFoundError`	필요한 패키지 미설치	`uv add 패키지명` 후 `uv run`
SSH `Permission denied (publickey)`	등록 키 / 사용자 / 키 파일 불일치	각 계정의 `authorized_keys`와 본인 개인키 확인
`furiosa-smi: command not found`	장치 관리 도구 미설치 또는 PATH 문제	관리자 사전 설치 상태 확인
RNGD 추론 실패	장치 권한, 런타임, 메모리, 모델 호환성 등	관리자와 상태 점검, 동시 실행 여부 확인

12. 종합 실습 과제

과제명: 사용자별 uv 프로젝트를 구성하고, 공용 RNGD 환경을 확인하라.

조별로 다음 작업을 완료한다.

- [ ] `std1`, `std2` 각각 SSH 접속
- [ ] `whoami`, `pwd`, `id`, `groups` 확인
- [ ] (권한 허용 시) `sudo whoami` 확인
- [ ] 각 계정에 `uv` 설치 및 버전 확인
- [ ] `uv init rngd-lab --python 3.12` 실행
- [ ] `uv python pin 3.12` 실행
- [ ] `uv add numpy pandas requests` 실행
- [ ] `uv sync`로 `.venv` 및 잠금 상태 점검
- [ ] `uv run python check_env.py` 실행

- [ ] 각 계정의 Python 경로와 의존성 비교
- [ ] `furiosa-smi info` 조회 (사전 설치된 환경인 경우)